

Assessment Task 2: Introduction and Methodology

Cryptanalysis of SEPAR, a Lightweight IoT Cipher

Author: Noah Ostle - 24740589
Degree: Bachelors of Computing Science w. Honours
Research field: Cryptanalysis
Supervisor: Dr. Luke Mathieson
School: University of Technology Sydney, Faculty of Engineering and IT
Date: May 2026

Contents

1	Introduction	2
1.1	Project motivation	2
2	Background and Related Work	4
2.1	SEPAR as a hybrid lightweight construction	4
2.2	Lessons from Hummingbird-1	4
2.3	Other broken lightweight or constrained ciphers	5
2.4	Comparable designs that <i>have</i> withstood broader scrutiny	5
3	Preliminary SEPAR Observations	6
3.1	A complete one-word codebook is feasible	6
3.2	The stage cascade separation	6
3.3	Quotient structure	6
3.4	Triangularity and weak diffusion	7
3.5	Differential properties	7
3.6	Matched-context	8
4	Research Aim and Questions	8
4.1	Aim	8
5	Methodology	8
5.1	Stage 1: Design review	8
5.2	Stage 2: Adversarial model and public observables	9
5.3	Stage 3: Proof and testing of individual weakness points	9
5.4	Stage 4: Connecting the components	9
5.5	Stage 5: Experimental validation	10
5.6	Stage 6: Critical review	10
6	Expected Contribution	10

1 Introduction

1.1 Project motivation

The Internet of Things has changed the practical setting in which symmetric cryptography is expected to operate. Many contemporary systems contain sensors, RFID-like devices, smart locks, medical monitors, industrial controllers, small wireless nodes, and other embedded components that process sensitive data while operating under strict limits on memory, gate count, clock speed, energy, and latency. These devices may be inexpensive and physically constrained, but the information they protect is not necessarily low value. A networked card reader in a government building, a wireless actuator in an industrial facility, or a medical telemetry component in a hospital can create serious security consequences with severe real world implications if its cryptography fails. The design problem is therefore not simply to make encryption fast. It is to preserve a meaningful security margin while making the algorithm implementable on hardware and software platforms that cannot afford the cost profile of conventional general-purpose primitives.

Lightweight cryptography exists to address this tension. It usually obtains efficiency by simplifying one or more parts of a design: using small S-boxes, small blocks, simple linear layers, reduced memory, compact key schedules, or repeated low-cost round functions. None of these choices is automatically insecure. In fact, well-designed lightweight ciphers can be strong precisely because their designers understand the cost of every component and compensate for each simplification elsewhere. The risk is that a construction may appear to have a large key size or pass ordinary statistical tests while still containing exploitable structure at the level of its state update, round composition, key schedule, or implementation-specific construction. The cryptanalytic task is to determine whether the efficiency choices form a coherent design or whether they expose a shortcut resultant from one or more of its efficient design factors.

SEPAR is a proposed lightweight hybrid encryption algorithm for IoT environments [1]. It is presented as combining stream-cipher-style stateful operation with block-cipher-style transformations. The design uses 16-bit words, a 128-bit initialization vector, eight internal 16-bit state words, an additional 16-bit LFSR-related state component, and eight stage-local 16-bit encryption components keyed by a 256-bit master key. Each stage is built from 4-bit S-boxes, XOR, rotations, and modular addition. The authors argue that this construction gives a favourable trade-off between security, performance, and implementation cost on 8-bit, 16-bit, and 32-bit platforms. The design paper also includes explicit resistance claims against several standard classes of attack, including differential, linear, algebraic, slide, related-key, interpolation, high-order differential, key-collision, and statistical attacks [1]. A later designer-authored paper further discusses SEPAR's resistance to differential cryptanalysis [2].

This project is motivated by preliminary analysis that has noticed several unusual behaviours in the SEPAR implementation. These behaviours do not, on their own, prove that SEPAR is insecure. Rather, they suggest that the cipher has structural features worth investigating carefully:

1. A 16-bit block size
2. The encryption path being a series of translated stage permutations
3. Early experiments suggesting quotient maps in a round permutation based on some bits of the stage key being independent of the output
4. triangular or weak-diffusion effects in parts of the implemented linear layer

5. Possible differential behavior
6. Matched-context ideas given the symmetry under Enc/Dec rounds

The primary task of the project is to determine whether there are any additional weaknesses of the construction, then whether these individual observations survive rigorous proof or statistically strong empirical testing, and whether they can be combined into a meaningful cryptanalytic result.

The central **research problem** is therefore:

Does the SEPAR ciphers design expose structural or statistical properties that, under a clearly defined adversarial model, allow an attacker to distinguish, weaken, or recover information about the cipher more efficiently than generic exhaustive methods?

This thesis will begin from a set of suspicious design and implementation features, then test them. If the suspected weaknesses can be formalised and connected, the project may produce a practical attack or a theoretical shortcut. If they cannot, the project should still produce a useful negative result by explaining which apparent weaknesses were artefacts, which were real but insufficient, and which parts of SEPAR resisted the analysis, satisfying the lack of critical discussion on SEPAR in the literature, and contributing to the third-party criticism focused literature of cryptography as a whole.

The history of symmetric cryptanalysis shows that a cipher can fail for reasons that are deeper, and more specific to an individual construction than any statistical scores (like the ones SEPAR’s authors rely on) would suggest. Differential cryptanalysis showed that non-random propagation of input differences can be converted into key information when enough chosen or known plaintexts are available [10, 11]. Linear cryptanalysis showed that small correlations between plaintext, ciphertext, and key-dependent internal values can likewise be accumulated into a key-recovery attack [12, 13]. Slide attacks exploit repetition and weak round variation rather than a conventional differential trail [14]. Related-key attacks exploit structure in the way different keys produce related computations [15]. Invariant and subspace-style attacks show that a cipher can preserve a hidden low-dimensional structure even when local components look nonlinear [16]. These attack families are different, but they share a common lesson: the attacker must closely examine the cipher’s structure, understanding how it works and which vulnerability classes it may be vulnerable to, in order to devise a targeted attack, rather than just blindly applying the most popular statistical tests.

This lesson is especially important in lightweight design. A compact cipher may use a small S-box with good local metrics, but the global construction can still be weak. A primary example of this is the family of subspace attacks, wherein the cipher would be *on average* random and secure on paper, but there exists a tiny subspace with negligible size on the order of the statistical test, that once identified can be used to reliably break the cipher. A design may advertise a large key, but use a key schedule or stage decomposition that lets the attacker test pieces of the key independently. A stateful cipher may contain many state bits, but leak or otherwise expose state bits. A statistical test suite may report good randomness on output streams, while a chosen-input attacker exploits exact algebraic or combinatorial structure that those tests were never designed to measure. For this reason, the methodology of this project emphasises exact analysis, reproducible experiments, and analysis specific to SEPAR’s construction.

2 Background and Related Work

2.1 SEPAR as a hybrid lightweight construction

SEPAR is described by its designers as a hybrid algorithm that draws on both block-cipher and stream-cipher ideas [1]. Its encryption of a 16-bit word passes through eight `ENC_BLOCK` calls. In the public implementation, if the eight state words are denoted $s_1, \dots, s_8$ and the eight stage-local permutations are denoted $E_1, \dots, E_8$, then one reset one-word encryption can be modelled as

$$F_{IV} = B_8^{(s_8)} \circ B_7^{(s_7)} \circ \dots \circ B_1^{(s_1)}, \quad B_i^{(s_i)}(x) = E_i(x + s_i),$$

where addition is modulo 2^{16} . This representation is important because it separates the stage-local keyed permutation E_i from the IV-dependent state translation s_i . The public ciphertext word is not merely the output of one monolithic black box; it is the output of a cascade whose internal structure may or may not remain hidden after a complete codebook has been collected.

The stage-local block itself is built from repeated S-box, linear-layer, and key-addition operations. A natural implementation model treats a stage key as a pair of 16-bit words

$$K_i = (k_{i,0}, k_{i,1}) \in \mathbb{Z}_{2^{16}} \times \mathbb{Z}_{2^{16}},$$

with two additional derived 16-bit subkeys produced from rotations and an S-box expression. The corresponding stage permutation can then be written as a composition of S-box layers, linear layers, and XOR key additions. This abstraction is useful because it makes clear that each stage has a 32-bit local key and that the whole 256-bit key is partitioned into eight such stage keys. A major question for this project is whether that partition is safely hidden by the cascade and state update, or whether the public behaviour leaks enough structure to test stages separately.

The design specification argues that SEPAR’s state and initialization choices help compensate for the 16-bit block size. It also compares SEPAR against other lightweight ciphers, including Hummingbird and PRESENT, in terms of performance on constrained microcontrollers [1]. This is a natural design comparison, but it also motivates a cryptanalytic comparison. The important question for this thesis is not only whether SEPAR is fast, but whether the particular way it combines small blocks, repeated stages, state updates, and simple operations prevents stage-level analysis.

2.2 Lessons from Hummingbird-1

Hummingbird-1 is the closest motivating example for this project because it is also a hybrid lightweight design intended for constrained devices [6]. Like SEPAR, it was motivated by environments such as RFID tags, smart cards, and wireless sensor nodes, and it combined a small-block internal structure with stateful operation. The general design idea was not unreasonable: use compact 16-bit components and internal state to obtain a low-cost authenticated encryption mechanism for devices that cannot afford larger conventional primitives.

The subsequent cryptanalysis of Hummingbird-1 is therefore highly relevant. Saarinen’s cryptanalysis showed that the first Hummingbird design contained exploitable weaknesses despite its intended hybrid structure [7]. The significance for this thesis is not that SEPAR and Hummingbird-1 will share the same weaknesses. They do not. The significance is that internal state and a hybrid block-stream presentation do not automatically neutralise the risks created by a very small public data path. If the attacker can choose inputs, control or repeat initialization conditions, and observe enough of the small-domain behaviour, then stateful wrapping

may not prevent a structural attack.

Hummingbird-1 is also a useful caution against relying too heavily on high-level claims such as "the cipher has many internal states" or "the block transformation has nonlinear S-boxes", which we observe in the on SEPAR as well. Those statements may be true and still not address the property that the attack exploits. In SEPAR, the preliminary direction is similarly construction-specific. The suspicious behaviour is not simply that the word size is 16 bits, nor simply that the stage functions use 4-bit S-boxes. The concern is the combination of a small codebook, translated stage composition, and 32 bit key schedule may exacerbate other 'bad but not fatal' statistical weaknesses, like, weak-diffusion effects, differentials, etc.

For the proposed thesis, Hummingbird-1 therefore provides a model of how to frame the background argument. It supports the research rationale: lightweight hybrid ciphers deserve exact structural analysis because previous designs in this family have been broken when their small components interacted in unexpected ways.

2.3 Other broken lightweight or constrained ciphers

Several other lightweight or constrained-environment ciphers reinforce the same methodological lesson. KeeLoq, for example, is a 32-bit block cipher with a 64-bit key that was widely used in passive entry and remote keyless entry systems. Bogdanov proposed a key-recovery attack with complexity around 2^{52} KeeLoq encryptions [8]. The relevance to SEPAR is the role of small blocks and repeated structure. KeeLoq was not broken because its ciphertexts failed a generic randomness test in a simple way. It was attacked by exploiting properties of the actual block-cipher construction and its operational setting.

KTANTAN provides a different example. It is a family of small, hardware-oriented block ciphers designed for low-resource applications. Bogdanov and Rechberger gave meet-in-the-middle attacks on the full KTANTAN family that exploit weaknesses in the bitwise key schedule and have very low data requirements [9]. This is relevant because SEPAR's attack surface may also involve separating key material by stage. They show how a cipher with a large nominal key can become vulnerable if enough of the computation depends on separable subsets of key bits.

The broader point is that lightweight cryptanalysis often turns on the exact trade-off made by the designer. Reducing area or memory tends to encourage repetition, narrow components, and simple key injection (all observed in the SEPAR ciphers GitHub implementation). These features are attractive to implementers, but they are also attractive to cryptanalysts because they can preserve structure across the cipher. PRESENT's designers explicitly discuss this tension: compact hardware favours repetition, while cryptanalysts also favour repetition because it can allow structures to propagate across rounds [4]. SEPAR should therefore be assessed at this level. The question is not whether its local S-boxes have acceptable individual metrics, but whether the complete implemented construction leaves a repeated or separable structure visible to an attacker.

2.4 Comparable designs that *have* withstood broader scrutiny

It is also important to compare SEPAR with lightweight designs that are not generally treated as cautionary broken examples. PRESENT and GIFT are useful reference points [4, 5]. Both are lightweight block-cipher families built around small S-boxes and bit permutations, and both have attracted substantial public cryptanalysis. The relevance is not that any cipher can be proven secure forever by surviving a number of papers. Rather, these designs illustrate how a mature lightweight proposal normally develops: the design is specified precisely, the security

argument is connected to known attack families, reduced round analysis accumulates over time, and the remaining full-round security claim is understood in relation to explicit security margins, giving confidence (in place of hard proof) that these designs are secure.

This comparison helps clarify what is missing for SEPAR. SEPAR has a published specification, implementation, and designer-authored security discussion, but it does not appear to have the same depth of independent external analysis. Its hybrid structure also differs from classical SPN block-cipher designs such as PRESENT and GIFT. A direct comparison of cycle counts or memory usage is therefore not enough. If SEPAR’s unusual state and stage structure is the main source of its claimed security, then that structure must itself be analysed.

3 Preliminary SEPAR Observations

3.1 A complete one-word codebook is feasible

For any fixed key and IV under reset access, SEPAR’s interface gives a 16-bit permutation. A full chosen-plaintext codebook costs exactly 2^{16} encryptions. In many conventional settings, such a table would still be too small to recover a large key, because a secure block cipher with a large key should look like an independently keyed random permutation on that domain. In SEPAR, however, the table is not produced by a single ideal 16-bit primitive. It is produced by eight translated stage permutations with a structured key partition.

This observation motivates the first line of investigation. The codebook should be treated as a public object:

$$F_{IV} : \mathbb{Z}_{2^{16}} \rightarrow \mathbb{Z}_{2^{16}}.$$

The research task is to determine whether the family of such objects, as the IV varies, contains detectable stage information. If not, then the small domain may be a performance concern but not an exploitable weakness under this model. If it does, then the small block size may enable exact table cryptanalysis.

3.2 The stage cascade separation

If outer stages could somehow be recognised, and if the corresponding state translation could be determined or cancelled, then the inverse of that stage could be applied to a public codebook. In the idealised notation

$$F_{IV} = B_8^{(ss)} \circ R_7,$$

solving the outer component would transform the observed table into a translated version of the seven-stage inner cascade R_7 . In principle, repeating this idea could reduce the problem one stage at a time.

Several major questions must be answered before we can confirm this works; Does any observable feature of the codebook identify a stage key more efficiently than brute force? Can the unknown state translation be recovered, cancelled, or made irrelevant to the score being used? Does a peel preserve enough information to attack the next stage? Does the process branch, and if so, what complexity bound does incur? Much work remains to be done in answering these questions, however given the ciphers design, and the implications of stage by stage peeling, this would be a very desirable property to have for any proposed attack.

3.3 Quotient structure

One of the most interesting preliminary observations is the apparent structure of high-byte quotient information. Let

$$H(y) = \left\lfloor \frac{y}{2^8} \right\rfloor$$

be the high byte of a 16-bit encryption. For a stage key K_i , define the high-byte quotient map

$$Q_{i,K_i}(h) = H(E_i(256h)), \quad h \in \mathbb{Z}_{256},$$

and a projected 4-bit map

$$q_{i,K_i}(h) = Q_{i,K_i}(h) \bmod 16.$$

Preliminary testing suggests that these quotient views may reveal information about stage-local behaviour that is less visible in the full 16-bit permutation. In testing, it was seen that some stage key bits appear invisible when examining a single stage permutation, although it remains to be seen if this information can be seen from an attackers perspective rather than a testing harness (with internal visibility over the cipher), and whether or not this map will vary from key to key.

This could be useful if the quotient is invariant between keys, and can be abused from an adversarial position, as it will allow an attacker to extract information about the reduced quotient, rather than the whole key, providing a more narrow candidate pool for a key guessing method than traditional bruteforce.

3.4 Triangularity and weak diffusion

Another promising observation concerns the implemented linear layer and possible triangular behaviour. Preliminary statistical analysis suggests that some parts of the `ENC.Block` computation may allow upper-byte information to evolve with limited influence from lower-byte information, at least before modular additions introduce carries. If true, this would be cryptanalytically important because it could reduce a 16-bit problem to smaller coupled subproblems.

This needs further verification however - the state additions, S-box layers, repeated stages, and carries may destroy or hide the structure by the time the attacker observes the final ciphertext word, as opposed to the clean single enc block view we had during initial testing.

We should therefore test triangularity at three levels: inside the implemented linear layer, inside a single stage permutation, and inside the full translated stage cascade. The goal is to determine whether the triangular effect is a real invariant, a useful source of statistical bias, or a negligible affect by the time the ciphertext reaches the attacker.

3.5 Differential properties

Preliminary experiments also suggest that some differential properties may be worth examining. The `SEPAR` specification discusses differential properties of `ENC.Block` and reports resistance claims [1, 2]. However, differences in a cipher that uses both XOR and modular addition must be defined carefully. XOR differences, modular additive differences, and differences before or after a state translation are not interchangeable.

One reason additive differences are promising is that output translations cancel. If a public current table has the form

$$T(x) = R(x) + s$$

for an unknown output translation s , then modular additive differences satisfy

$$T(x + \delta) - T(x) = R(x + \delta) - R(x).$$

This could make additive difference scores useful for recognising a peeled or partially peeled table even when a state word remains unknown. The lack of specificity on the authors part warrants more investigation given this aspect of the ciphers design.

3.6 Matched-context

A further line of investigation is the use of matched contexts. Because SEPAR is stateful, a direct comparison between two encryptions may be misleading if the internal state differs. However, if a chosen plaintext prefix can recreate a matching state context before a target word, then the next-word encryption and decryption behaviour may be compared more cleanly. This idea may help distinguish true structural behaviour from artefacts caused by changing state.

The matched-context direction is still exploratory, but the given the ciphers 'symmetricality' about encryption and decryption, it is a property worth investigating under the resettable state model, as it could provide an avenue for attackers to ignore the state through clever algebra.

4 Research Aim and Questions

4.1 Aim

Aim: *To analyse the SEPAR cipher and determine whether its lightweight construction contains any structural or statistical weaknesses that can be developed into a cryptanalytic attack.*

The goal is first to understand the cipher accurately, then to prove or refute the suspected weakness points, and only then to determine whether those points can be assembled into a meaningful attack. Success in research aim may therefore have several possible outcomes. It may produce a complete attack, a partial attack, just a statistically significant weakness of the cipher, or even a well documented negative result showing that the preliminary observations do not combine into a practical break.

The aim therefore in sum is to provide a cryptanalytic evaluation of the SEPAR cipher, as is the goal of all cryptanalytic research - to add to the existing literature, and either provide confidence for, or reason against using particular constructions, both in production ciphers, and for the purpose of future cipher designs.

5 Methodology

5.1 Stage 1: Design review

The first stage is to build a precise model of the cipher actually being analysed. This requires reading the SEPAR design paper, the later differential-cryptanalysis paper, and the public C implementation together [1, 2]. The output of this stage will be a clean mathematical description of the implementation, including the state model, IV handling, stage-local encryption function, state update rule, and reset behaviour.

This step is necessary because lightweight ciphers are often described at a high level, while attacks depend on exact implementation details. For example, we must distinguish the abstract design from the published GitHub C code behaviour, and identify whether any apparent weakness comes from the intended high level design as described in plain English by the authors, the proposed mathematical implementation and statistical properties of mathematical objects like the S-Box or linear layer, or the published C implementation itself.

Another key part of this stage is a second focused attempt to identify points of interest for more rigorous testing. Another analysis pass will help pick up any further potential weaknesses that warrant further testing in stage 3.

5.2 Stage 2: Adversarial model and public observables

The next stage is to define the adversarial model. The working model may include chosen plaintext, chosen IV, and state reset between oracle calls, but each capability must be justified under an adversarial model. It should also identify which observables exist under which adversarial models. For example, a full-codebook observation may require reset access, while a matched-context experiment may require chosen prefixes. The model should be stated before any attack claim is made.

For each fixed IV, the primary public observable will be the complete codebook

$$\mathcal{C}_{IV} = \{(x, F_{IV}(x)) : x \in \mathbb{Z}_{2^{16}}\}.$$

The methodology will treat this table as the raw data object from which all derived tables, inverse tables, quotient maps, and difference scores are computed.

5.3 Stage 3: Proof and testing of individual weakness points

The third stage is to test each preliminary weakness independently;

1. For stage structure, we will formalise the translated-stage cascade and determine what information remains visible after one or more stages are composed.
2. For quotient structure, we will define the exact high-byte and projected quotient maps, then test whether they reliably expose key information.
3. For triangularity, we will analyse the implemented linear layer and trace whether limited-dependency behaviour survives through S-boxes, key additions, carries, and stage composition.
4. For differential properties, we will perform statistical testing and measure biases.
5. For matched contexts, the project will build experiments that compare encryption and decryption behaviour under controlled state conditions.
6. Any further weaknesses identified by stage 1.

Where a property is exact, the thesis will aim to prove it from the designs definitions. Where a property is empirical, the thesis will report the sampling method, random seeds, number of trials, and observed failure cases.

5.4 Stage 4: Connecting the components

After stage 3, the results should be taken, and used to determine if an attack exists against the cipher. This will be composed of an analysis on the ciphers structure, given the observables from stage 3, as well as an attack designing phase.

We aim to put forward the most complete attack possible on the cipher, including compromises of integrity, and confidentiality of ciphertext, or attacks against the key. Failing this, we will put forward either the pure observable results from stage 3, or a negative result consisting of documentation of failed attacks, and analysis on how the ciphers design mitigated them.

5.5 Stage 5: Experimental validation

This stage will focus on the validation of any confirmed primitives and any candidate attack paths. If the attack is deterministic, it must be proven as a consequence of the cipher design. If the attack is statistical, it must be proven empirically. This will include true random seed based generation of keys, plaintexts, and IV's, as well as a Monte Carlo calculation for the probability of success.

5.6 Stage 6: Critical review

The final stage is an analysis of the attack. This will feature a calculation of the computational complexity bound of the attack, as well as a discussion on the implications on the adversarial model, the attacks effectiveness and reliability, and the aspects of the ciphers construction that led to its weaknesses, or lack thereof.

6 Expected Contribution

The expected contribution is an academic cryptanalysis of the SEPAR cipher that either adds confidence to the ciphers security in the case of a negative result, or proposes a 'from first principles' proof, or empirical evidence for a weakness, with a discussion of any implications for the ciphers use. In this way, my work will directly comment on the justification for this ciphers use in production IoT systems, as proposed by the authors.

The broader contribution is defined by the natural goal of cryptanalytic research en masse - to, through much academic scrutiny, and rigorously well informed trial and error, provide reasonable confidence for the security of cryptographic designs. Putting forward my cryptanalysis (either a positive or negative result) will contribute to the broader existing literature, serving as a discussion on what aspects of a ciphers design make it secure, and which can lead to weaknesses, with specific relevance to lightweight hybrid designs.

References

- [1] A. Vahi and S. Jafarali Jassbi. SEPAR: A new lightweight hybrid encryption algorithm with a novel design approach for IoT. *Wireless Personal Communications*, 114:2283–2314, 2020.
- [2] A. Vahi and M. Mirnia. On the differential cryptanalysis of SEPAR cipher. arXiv preprint arXiv:2106.12638, 2021.
- [3] A. Y. Poschmann. *Lightweight Cryptography: Cryptographic Engineering for a Pervasive World*. PhD thesis, Ruhr-University Bochum, 2009.
- [4] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsoe. PRESENT: An ultra-lightweight block cipher. In *Cryptographic Hardware and Embedded Systems - CHES 2007*, pages 450–466. Springer, 2007.
- [5] S. Banik, S. K. Pandey, T. Peyrin, Y. Sasaki, S. M. Sim, and Y. Todo. GIFT: A small present. In *Cryptographic Hardware and Embedded Systems - CHES 2017*. Springer, 2017.
- [6] D. W. Engels, X. Fan, G. Gong, H. Hu, and E. M. Smith. Hummingbird: Ultra-lightweight cryptography for resource-constrained devices. In *Financial Cryptography and Data Security*, pages 3–18. Springer, 2010.

- [7] M.-J. O. Saarinen. Cryptanalysis of Hummingbird-1. In *Fast Software Encryption*, pages 328–341. Springer, 2011.
- [8] A. Bogdanov. Cryptanalysis of the KeeLoq block cipher. Cryptology ePrint Archive, Paper 2007/055, 2007.
- [9] A. Bogdanov and C. Rechberger. A 3-subset meet-in-the-middle attack: Cryptanalysis of the lightweight block cipher KTANTAN. Cryptology ePrint Archive, Paper 2010/532, 2010.
- [10] E. Biham and A. Shamir. Differential cryptanalysis of DES-like cryptosystems. *Journal of Cryptology*, 4:3–72, 1991.
- [11] H. M. Heys. A tutorial on linear and differential cryptanalysis. *Cryptologia*, 26(3):189–221, 2002.
- [12] M. Matsui. Linear cryptanalysis method for DES cipher. In *Advances in Cryptology - EUROCRYPT '93*, pages 386–397. Springer, 1993.
- [13] M. Matsui and A. Yamagishi. A new method for known plaintext attack of FEAL cipher. In *Advances in Cryptology - EUROCRYPT '92*, pages 81–91. Springer, 1993.
- [14] A. Biryukov and D. Wagner. Slide attacks. In *Fast Software Encryption*. Springer, 1999.
- [15] E. Biham. New types of cryptanalytic attacks using related keys. *Journal of Cryptology*, 7(4):229–246, 1994.
- [16] J. Guo, J. Jean, I. Nikolic, K. Qiao, Y. Sasaki, and S. M. Sim. Invariant subspace attack against Midori64 and the resistance criteria for S-box designs. *IACR Transactions on Symmetric Cryptology*, 2016(1):33–56, 2016.
- [17] G. Leander and A. Poschmann. On the classification of 4 bit S-boxes. In *Arithmetic of Finite Fields - WAIFI 2007*. Springer, 2007.
- [18] M.-J. O. Saarinen. Cryptographic analysis of all 4 x 4-bit S-boxes. In *Selected Areas in Cryptography*. Springer, 2011.
- [19] L. E. Bassham III et al. A statistical test suite for random and pseudorandom number generators for cryptographic applications. NIST Special Publication 800-22 Revision 1a, 2010.